WEEK 15 — Unit 6: CI/CD with Jenkins

Topic: Jenkins Foundations, Pipelines, Stages, Docker Integration, Maven Integration & CI/CD Deployment Flows Course: INT 332 – DevOps

---

Note 1: Jenkins Foundations — Architecture, Installation, Plugins & Security

What is Jenkins?

Jenkins is an open-source automation server written in Java. It is one of the oldest and most widely used CI/CD tools in the industry. Jenkins automates the repetitive tasks involved in building, testing, and deploying software. Unlike GitHub Actions, which is tightly coupled to GitHub, Jenkins is platform-agnostic — it can work with any version control system, any build tool, and any deployment target.

Jenkins follows a server-based model where a central Jenkins server orchestrates all automation tasks, either running them itself or delegating them to agent machines.

Jenkins Architecture — Master/Agent Model

The Master (Controller)

The Jenkins Master, now officially called the Controller, is the central brain of Jenkins. It is responsible for:

- Storing all configuration, job definitions, and build history
- Scheduling build jobs and deciding when and where they run
- Serving the Jenkins web UI for users to interact with
- Managing plugins, credentials, and security settings
- Distributing work to agents

The Master itself should not run heavy build jobs — it should focus on orchestration. Running builds on the Master is considered bad practice because it can overload the server and create security risks.

Agents (Workers)

An agent is a machine — physical, virtual, or containerised — that is connected to the Jenkins Master and executes build jobs on its behalf. Agents can run on Linux, Windows, or macOS.

Each agent has one or more **executors** — slots that can run one build job at a time. An agent with 4 executors can run 4 jobs in parallel.

Agents are identified by **labels** — tags that describe their capabilities (e.g., linux, docker, java17, gpu). Jobs can specify which agent label they need, and Jenkins schedules them on a matching agent.

How the Master/Agent Model Works

- A developer pushes code to a repository
- Jenkins Master detects the event (via webhook or polling)
- Master evaluates which agent is available and matches the job's label requirement
- Master sends the job to the selected agent
- Agent clones the repository, runs the build steps, and reports results back to the Master
- Master stores the build logs, test reports, and artifacts
- Master sends notifications (email, Slack) based on build results

Why This Model Matters

- Horizontal scaling — add more agents when build queue grows
- Isolation — each agent can have different tools installed
- Security — sensitive production deployments can be restricted to specific trusted agents
- Platform diversity — run builds on Linux agents and tests on Windows agents simultaneously

---

Jenkins Installation Overview

Jenkins is distributed as a WAR file and can be run in several ways:

Option 1: Running as a standalone application

```
java -jar jenkins.war --httpPort=8080
```

Option 2: Installing as a system service (Ubuntu/Debian)

```
# Add Jenkins repository key
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key \
  | sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null
```

```
# Add repository
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ \
  | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

# Install
sudo apt update
sudo apt install jenkins

# Start and enable
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Option 3: Running Jenkins in Docker

```
docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts
```

Port 8080 is the Jenkins web UI. Port 50000 is used for agent communication (JNLP protocol).

After installation, Jenkins is accessible at http://localhost:8080. The first-time setup wizard asks for an initial admin password found at /var/jenkins_home/secrets/initialAdminPassword.

Jenkins UI Overview

The Jenkins web UI has the following main sections:

- **Dashboard** — shows all jobs, their last build status, and build history
- **New Item** — creates a new job (Freestyle, Pipeline, Multibranch, etc.)

- **Manage Jenkins** — central configuration: system settings, plugins, credentials, agents, security
- **Build Queue** — shows jobs waiting to be executed
- **Build Executor Status** — shows currently running builds and which executor/agent is running them
- **People** — manages user accounts
- **Build History** — global view of recent builds across all jobs

Each job has its own page showing:

- Build history with status (green = success, red = failure, yellow = unstable)
- Console output for each build
- Test results and artifact downloads
- Configuration settings

Plugin Management

What are Plugins?

Jenkins has a minimal core — most functionality is provided through plugins. There are over 1,800 official plugins covering version control systems, build tools, cloud providers, notification systems, security integrations, and much more.

Managing Plugins

Navigate to **Manage Jenkins → Plugins** to access:

- **Available plugins** — browse and install new plugins
- **Installed plugins** — view and manage currently installed plugins
- **Updates** — plugins with available updates
- **Advanced** — manually upload plugin .hpi files, configure update sites

Essential Plugins for a Java/Docker CI Pipeline

Plugin	Purpose
Git Plugin	Integrates with Git repositories
GitHub Plugin	GitHub webhook integration and status updates

Plugin	Purpose
Maven Integration Plugin	Adds Maven build steps and global Maven config
Pipeline Plugin	Enables Jenkinsfile-based pipelines
Docker Pipeline Plugin	Provides docker DSL inside Jenkinsfiles
Docker Plugin	Manages Docker agents
Credentials Plugin	Securely stores usernames, passwords, SSH keys, tokens
JUnit Plugin	Publishes JUnit test results in the UI
Jacoco Plugin	Publishes code coverage reports
Email Extension Plugin	Sends rich email notifications
Blue Ocean Plugin	Modern UI for Jenkins pipelines
SSH Agent Plugin	Provides SSH credentials for deployment steps

---

Security, Users & Roles

Enabling Security

By default, Jenkins may allow anonymous access. Security is configured at **Manage Jenkins → Security**.

Security Realm — controls how users authenticate:

- **Jenkins own user database** — built-in user management (simplest option)
- **LDAP** — integrate with corporate Active Directory or LDAP server
- **GitHub OAuth** — log in with GitHub accounts

Authorization Strategy — controls what authenticated users can do:

- **Anyone can do anything** — no restrictions (only for local dev/testing)
- **Logged-in users can do anything** — all authenticated users have full access
- **Matrix-based security** — fine-grained control; assign specific permissions to specific users or groups
- **Role-based strategy** (requires Role Strategy Plugin) — create named roles with specific permissions and assign users to roles

Creating Users

Manage Jenkins → Users → Create User Fill in username, password, full name, and email.

Role-Based Access Control with Role Strategy Plugin

After installing the Role Strategy Plugin, under **Manage Jenkins → Security → Authorization**, select **Role-Based Strategy**.

Then under **Manage Jenkins → Manage and Assign Roles**:

Create Roles:

- admin — all permissions
- developer — can build jobs, view console output, cancel builds; cannot configure jobs or manage Jenkins
- viewer — can only view job status and build history

Assign Roles:

- Assign users or groups to the appropriate role

This ensures that developers cannot accidentally change Jenkins configuration, and that junior team members cannot trigger production deployments.

Note 2: Jenkins Pipelines — Freestyle vs Pipeline, Declarative & Scripted Syntax, Jenkinsfile

Freestyle Jobs vs Pipeline Jobs

Freestyle Jobs

Freestyle jobs are the original Jenkins job type. They are configured entirely through the Jenkins UI — you click through forms to define build steps, post-build actions, triggers, and parameters. No code is written.

Advantages of Freestyle Jobs:

- Easy to set up for beginners — no coding required
- Good for simple, single-step tasks

Disadvantages of Freestyle Jobs:

- Configuration lives in Jenkins, not in the repository — not version controlled
- Difficult to implement complex logic (conditionals, loops, parallel steps)
- Hard to review and audit changes to job configuration

- Cannot easily be shared or reused across projects
- Does not scale well for complex multi-stage pipelines

Pipeline Jobs

Pipeline jobs define the entire build process as code in a file called Jenkinsfile, which lives in the repository root. This is the modern, recommended approach.

Advantages of Pipeline Jobs:

- Pipeline-as-code — versioned alongside application code in Git
- Supports complex logic — conditionals, loops, parallel stages, error handling
- Durable — survives Jenkins restarts; the pipeline state is preserved
- Reusable — shared pipeline libraries can be used across multiple projects
- Reviewable — pipeline changes go through the same pull request process as code changes

---

Declarative vs Scripted Pipeline Syntax

Jenkins supports two syntaxes for writing pipelines:

Declarative Pipeline Syntax

Declarative is the newer, more structured syntax. It uses a predefined structure with specific blocks and keywords. It is easier to read, write, and maintain for most use cases. This is the recommended syntax for beginners and most teams.

```
pipeline {
  agent any

  stages {
    stage('Build') {
      steps {
        echo 'Building...'
        sh 'mvn clean package'
      }
    }
```

```
    stage('Test') {
      steps {
        echo 'Testing...'
        sh 'mvn test'
      }
    }
    stage('Deploy') {
      steps {
        echo 'Deploying...'
      }
    }
  }
}
```

The top-level structure is always:

- pipeline { } — the root block
- agent — where to run
- stages { } — contains all stage blocks
- stage('Name') { } — a single stage
- steps { } — the actual commands inside a stage

Scripted Pipeline Syntax

Scripted is the older syntax. It uses a Groovy-based DSL and gives full programmatic control. It is more flexible but more complex and harder to read.

```
node {
  stage('Build') {
    sh 'mvn clean package'
  }
  stage('Test') {
    sh 'mvn test'
```

```
    }
    stage('Deploy') {
        sh 'echo Deploying...'
    }
}
```

The top-level block is node { } instead of pipeline { }. There is no enforced structure — you have complete freedom, which also means more room for error.

When to Use Which

- Use **Declarative** for most pipelines — it is cleaner, easier to understand, and has built-in support for post actions, options, and environment blocks
- Use **Scripted** only when you need advanced Groovy logic that declarative cannot handle — such as dynamic stage generation or complex conditional branching

---

Jenkinsfile Structure in Detail

Full Declarative Jenkinsfile

```
pipeline {
    // Where to run — 'any' means any available agent
    agent any

    // Pipeline-level options
    options {
        timeout(time: 1, unit: 'HOURS')       // Abort if pipeline runs > 1 hour
        buildDiscarder(logRotator(numToKeepStr: '10'))  // Keep only last 10 builds
        disableConcurrentBuilds()             // Don't run multiple builds at once
    }

    // Pipeline-level environment variables
    environment {
        APP_NAME = 'my-app'
```

```
    DOCKER_REGISTRY = 'docker.io/myusername'
    JAVA_HOME = '/usr/lib/jvm/java-17-openjdk'
  }

  // Build parameters — shown as form in UI when building manually
  parameters {
    string(name: 'BRANCH', defaultValue: 'main', description: 'Branch to build')
    booleanParam(name: 'RUN_TESTS', defaultValue: true, description: 'Run tests?')
    choice(name: 'ENVIRONMENT', choices: ['staging', 'production'], description: 'Deploy target')
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm   // Checks out the repository that triggered this build
      }
    }

    stage('Build') {
      steps {
        sh './mvnw clean package -DskipTests'
      }
    }

    stage('Test') {
      when {
        expression { params.RUN_TESTS == true }
```

```
    }
    steps {
        sh './mvnw test'
    }
    post {
        always {
            junit 'target/surefire-reports/*.xml'
        }
    }
}

stage('Package Docker Image') {
    steps {
        sh "docker build -t ${DOCKER_REGISTRY}/${APP_NAME}:${BUILD_NUMBER} ."
    }
}

stage('Push Docker Image') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'dockerhub-credentials',
            usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_PASS'
        )]) {
            sh "echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin"
            sh "docker push ${DOCKER_REGISTRY}/${APP_NAME}:${BUILD_NUMBER}"
        }
    }
```

```
    }

    stage('Deploy') {
        when {
            branch 'main'
        }
        steps {
            sh "echo Deploying ${APP_NAME} to ${params.ENVIRONMENT}..."
        }
    }
}

// Post actions run after all stages complete
post {
    success {
        echo 'Pipeline succeeded!'
        emailext subject: "Build Successful: ${JOB_NAME} #${BUILD_NUMBER}",
                 body: "The build succeeded.",
                 to: 'team@example.com'
    }
    failure {
        echo 'Pipeline failed!'
        emailext subject: "Build Failed: ${JOB_NAME} #${BUILD_NUMBER}",
                 body: "The build failed. Check console output.",
                 to: 'team@example.com'
    }
    always {
        cleanWs()   // Clean up workspace after every build
```

```
    }
  }
}
```

Key Jenkinsfile Concepts Explained

agent Directive

Specifies where the pipeline or a specific stage runs:

```
agent any                    // Any available agent
agent none                   // No global agent; each stage defines its own
agent { label 'linux' }      // Agent with the 'linux' label
agent { docker 'maven:3.9-jdk-17' } // Run inside a Docker container
agent {
  node {
    label 'linux'
    customWorkspace '/opt/builds/my-app'
  }
}
```

environment Block

Defines environment variables available throughout the pipeline or within a specific stage:

```
environment {
  APP_VERSION = '1.0.0'
  // Load a secret from Jenkins Credentials store
  DB_PASSWORD = credentials('db-password-credential-id')
}
```

when Directive

Controls whether a stage runs based on conditions:

```
stage('Deploy to Production') {
  when {
```

```
        branch 'main'                    // Only on main branch
    }
}

stage('Run Integration Tests') {
    when {
        expression { params.RUN_TESTS == true } // Based on parameter
    }
}

stage('Notify') {
    when {
        anyOf {
            branch 'main'
            branch 'develop'
        }
    }
}
```

post Block

Defines actions to run after a stage or the entire pipeline completes:

```
post {
    always { }      // Always runs
    success { }     // Runs only on success
    failure { }     // Runs only on failure
    unstable { }    // Runs if build is unstable (tests passed but with warnings)
    changed { }     // Runs if build result differs from previous build
    aborted { }     // Runs if build was manually aborted
}
```

parameters Block

Defines input parameters that appear as a form in the Jenkins UI when triggering a manual build:

```
parameters {
  string(name: 'VERSION', defaultValue: '1.0.0', description: 'Version to deploy')
  booleanParam(name: 'SKIP_TESTS', defaultValue: false, description: 'Skip tests?')
  choice(name: 'ENV', choices: ['dev', 'staging', 'prod'], description: 'Environment')
  password(name: 'API_KEY', defaultValue: '', description: 'API Key')
}
```

Access parameters in steps: ${params.VERSION}, ${params.SKIP_TESTS}

Jenkins Multibranch Pipeline

A Multibranch Pipeline automatically creates and manages a separate pipeline for every branch and pull request in a repository. When a new branch is created, Jenkins detects it and creates a pipeline for it automatically. When a branch is deleted, its pipeline is removed.

Setup:

- New Item → Multibranch Pipeline
- Configure the Branch Source (GitHub, GitLab, Bitbucket)
- Jenkins scans the repository and creates pipelines for all branches containing a Jenkinsfile

Behaviour:

- Each branch gets its own build history
- Pull requests get their own pipeline run (using the PR merge commit)
- BRANCH_NAME environment variable is automatically set in each pipeline
- Use when { branch 'main' } to restrict deployment stages to specific branches

---

Note 3: Pipeline Stages, Docker Integration, Maven in Jenkins & CI/CD Deployment Flows

Standard Pipeline Stages in Detail

Stage 1: Checkout Code from Git

```
stage('Checkout') {
    steps {
        // For a Multibranch Pipeline, checkout scm uses the triggering branch
        checkout scm

        // Or explicitly specify a repository
        git branch: 'main',
            credentialsId: 'github-credentials',
            url: 'https://github.com/myusername/my-app.git'
    }
}
```

After checkout, all source files are available in the workspace directory on the agent.

Stage 2: Build

```
stage('Build') {
    steps {
        sh './mvnw clean compile -B'
        // -B flag = batch mode (no interactive prompts, better for CI)
    }
}
```

Stage 3: Test

```
stage('Test') {
    steps {
        sh './mvnw test -B'
    }
    post {
        always {
            // Publish JUnit test results in Jenkins UI
            junit 'target/surefire-reports/*.xml'
```

```
        // Publish code coverage report
        jacoco(
            execPattern: 'target/jacoco.exec',
            classPattern: 'target/classes',
            sourcePattern: 'src/main/java'
        )
    }
  }
}
```

Stage 4: Package

```
stage('Package') {
  steps {
    sh './mvnw package -DskipTests -B'
    // skipTests because tests already ran in the Test stage

    // Archive the JAR as a Jenkins artifact
    archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
  }
}
```

archiveArtifacts makes the JAR downloadable from the Jenkins build page. fingerprint: true enables tracking which builds used this artifact.

Stage 5: Post Actions and Managing Artifacts

```
post {
  always {
    // Clean workspace to free disk space
    cleanWs()
  }
```

```
    success {
        // Archive artifacts only on success
        archiveArtifacts artifacts: 'target/*.jar'
    }
    failure {
        // Send failure notification
        mail to: 'devteam@example.com',
             subject: "FAILED: ${JOB_NAME} - Build #${BUILD_NUMBER}",
             body: "Build failed. See: ${BUILD_URL}"
    }
}
```

Docker and Jenkins Integration

Building Docker Images in Jenkins

```
stage('Build Docker Image') {
    steps {
        script {
            def imageName = "myusername/my-app:${BUILD_NUMBER}"
            sh "docker build -t ${imageName} ."
        }
    }
}
```

For Docker commands to work, the Jenkins agent must have Docker installed and the Jenkins user must have permission to run Docker commands:

```
sudo usermod -aG docker jenkins
```

Docker Inside Jenkins Agents

The Docker Pipeline Plugin provides a cleaner DSL for Docker operations:

```
pipeline {
```

```
  agent any


  stages {
    stage('Build inside Docker') {
      agent {
        docker {
          image 'maven:3.9.4-eclipse-temurin-17'
          args '-v $HOME/.m2:/root/.m2'  // Mount Maven cache
        }
      }
      steps {
        sh 'mvn clean package'
      }
    }
  }
}
```

This runs the Maven build inside a temporary Maven Docker container instead of requiring Maven to be installed on the agent. The container is automatically pulled, used for the stage, and removed after. The -v argument mounts the host's Maven cache into the container to avoid re-downloading dependencies.

Publishing Images to Docker Hub

```
stage('Push to Docker Hub') {
  steps {
    withCredentials([usernamePassword(
      credentialsId: 'dockerhub-creds',
      usernameVariable: 'DOCKER_USERNAME',
      passwordVariable: 'DOCKER_PASSWORD'
    )]) {
      sh '''
```

```
            echo "$DOCKER_PASSWORD" | docker login -u "$DOCKER_USERNAME" --password-stdin
            docker tag my-app:${BUILD_NUMBER} ${DOCKER_USERNAME}/my-app:${BUILD_NUMBER}
            docker tag my-app:${BUILD_NUMBER} ${DOCKER_USERNAME}/my-app:latest
            docker push ${DOCKER_USERNAME}/my-app:${BUILD_NUMBER}
            docker push ${DOCKER_USERNAME}/my-app:latest
        '''
      }
   }
}
```

withCredentials safely injects the Docker Hub username and password from Jenkins Credentials store into environment variables. The values are masked in console output.

Publishing Images to GHCR

```
stage('Push to GHCR') {
   steps {
      withCredentials([string(
         credentialsId: 'github-token',
         variable: 'GITHUB_TOKEN'
      )]) {
         sh '''
            echo "$GITHUB_TOKEN" | docker login ghcr.io -u myusername --password-stdin
            docker tag my-app:${BUILD_NUMBER} ghcr.io/myusername/my-app:${BUILD_NUMBER}
            docker push ghcr.io/myusername/my-app:${BUILD_NUMBER}
         '''
      }
   }
}
```

Jenkins and GitHub Integration

To trigger Jenkins builds automatically when code is pushed to GitHub:

Step 1: Install GitHub Plugin in Jenkins

Step 2: Create a GitHub Personal Access Token (PAT)

- Go to GitHub → Settings → Developer settings → Personal access tokens
- Generate token with repo and admin:repo_hook scopes

Step 3: Add the PAT to Jenkins Credentials

- Manage Jenkins → Credentials → Add Credentials
- Kind: Secret text; ID: github-token; Secret: paste the PAT

Step 4: Configure GitHub in Jenkins

- Manage Jenkins → System → GitHub section
- Add GitHub Server; select the credential

Step 5: Configure Webhook in GitHub

- Repository → Settings → Webhooks → Add webhook
- Payload URL: http://your-jenkins-url/github-webhook/
- Content type: application/json
- Events: Just the push event (or all events)

Now every push to the repository triggers a Jenkins build automatically.

Maven Installation and Configuration in Jenkins

Installing Maven in Jenkins

Manage Jenkins → Tools → Maven installations → Add Maven

- Name: Maven-3.9
- Check "Install automatically"
- Select version 3.9.4

Jenkins downloads Maven automatically and makes it available to pipelines.

Using the Configured Maven in Jenkinsfile

pipeline {

```
    agent any

    tools {
        maven 'Maven-3.9'     // Must match the name configured in Tools
        jdk 'JDK-17'          // Must match the name configured in Tools
    }

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
    }
}
```

The tools block tells Jenkins to add the specified Maven and JDK to the PATH before running pipeline steps.

Running Maven Builds in Pipelines

```
stage('Build') {
    steps {
        sh 'mvn clean package -B -Dmaven.test.failure.ignore=false'
    }
}

stage('Test with Report') {
    steps {
        sh 'mvn test -B'
    }
```

```
    post {
        always {
            junit '**/target/surefire-reports/*.xml'
        }
    }
}

stage('Code Coverage') {
    steps {
        sh 'mvn verify -B'
    }
    post {
        always {
            jacoco execPattern: '**/target/jacoco.exec'
        }
    }
}
```

Jenkins CI/CD Deployment Flows

Triggering Builds

Method 1: pollSCM Jenkins periodically checks the repository for new commits and triggers a build if changes are found.

```
pipeline {
    agent any

    triggers {
        pollSCM('H/5 * * * *')   // Poll every 5 minutes
    }
```

```
  stages {
    stage('Build') {
      steps {
        sh 'mvn clean package'
      }
    }
  }
}
```

H/5 * * * * uses the H (hash) syntax which distributes polling load across Jenkins — instead of all jobs polling at exactly minute 0, 5, 10..., they are spread across different seconds to reduce server load.

Polling is less efficient than webhooks — it creates unnecessary load and has up to 5 minutes of delay. Use webhooks in production.

Method 2: Webhook (Preferred) As described in the GitHub integration section, a webhook sends an immediate trigger to Jenkins the moment code is pushed, with zero delay.

Jenkins Agents — SSH/SFTP/Container-Based

SSH Agent: Jenkins connects to remote servers via SSH to execute deployment commands.

```
stage('Deploy via SSH') {
  steps {
    sshagent(credentials: ['server-ssh-key']) {
      sh '''
        ssh -o StrictHostKeyChecking=no user@production-server \
        "docker pull myusername/my-app:latest && \
         docker stop my-app || true && \
         docker rm my-app || true && \
         docker run -d --name my-app -p 8080:8080 myusername/my-app:latest"
      '''
    }
```

```
    }
}
```

SFTP for File Transfer:

```
stage('Deploy via SFTP') {
    steps {
        sshagent(credentials: ['server-ssh-key']) {
            sh 'scp target/my-app.jar user@server:/opt/apps/my-app.jar'
            sh 'ssh user@server "systemctl restart my-app"'
        }
    }
}
```

Container-Based Agents: Jenkins can spin up Docker containers as agents dynamically — when a job starts, a fresh container is created; when it ends, the container is destroyed.

```
pipeline {
    agent {
        docker {
            image 'maven:3.9.4-eclipse-temurin-17-alpine'
            label 'docker-agent'
            args '--network host'
        }
    }
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
    }
```

}

Pipeline Libraries (Shared Libraries)

For large organisations with many Jenkins pipelines, shared libraries allow common pipeline logic to be defined once and reused across multiple Jenkinsfiles.

A shared library is a Git repository with a specific structure:

```
shared-library/
├── vars/
|   ├── buildJavaApp.groovy      # Global variable / function
|   └── deployToServer.groovy
└── src/
    └── org/example/
        └── BuildHelper.groovy   # Groovy class
```

Configure the library in **Manage Jenkins → System → Global Pipeline Libraries**.

Using a shared library in a Jenkinsfile:

```
@Library('my-shared-library') _


pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                buildJavaApp()    // Calls the shared buildJavaApp.groovy function
            }
        }
        stage('Deploy') {
            steps {
                deployToServer(env: 'staging')
            }
```

```
    }
  }
}
```

This approach eliminates copy-pasting the same pipeline logic across dozens of repositories.

Jenkins Backup and Restore

What Needs to be Backed Up

Jenkins stores all its data in the Jenkins home directory ($JENKINS_HOME, typically /var/lib/jenkins on Linux). This includes:

- Job configurations (jobs/ directory)
- Build history and logs (jobs/<job-name>/builds/)
- Plugin installations (plugins/)
- Credentials and secrets (secrets/)
- System configuration (config.xml)

Backup Strategy

Method 1: Backup the entire JENKINS_HOME directory

tar -czf jenkins-backup-$(date +%Y%m%d).tar.gz /var/lib/jenkins

Method 2: Using the ThinBackup Plugin The ThinBackup plugin provides scheduled backups through the Jenkins UI:

- Install ThinBackup Plugin
- Manage Jenkins → ThinBackup → Settings
- Configure backup directory, schedule, and retention

Restore

To restore from a backup:

sudo systemctl stop jenkins

sudo tar -xzf jenkins-backup-20240101.tar.gz -C /

sudo systemctl start jenkins

Pipeline Best Practices

1. Always use Declarative Syntax unless Scripted is specifically needed Declarative is easier to read, maintain, and has better built-in support for common patterns.

2. Store the Jenkinsfile in the repository root This keeps pipeline definition versioned with the code and makes it auditable through pull requests.

3. Use agent none at the top level with per-stage agents This avoids holding an agent idle during stages that don't need it (like approval wait stages).

4. Never hardcode credentials Always use withCredentials and the Jenkins Credentials store. Never put passwords, tokens, or keys directly in the Jenkinsfile.

5. Use cleanWs() in the post block Always clean the workspace after a build to prevent stale files from affecting future builds and to save disk space.

6. Keep stages focused and single-purpose Each stage should do one thing — Checkout, Build, Test, Package, Push, Deploy. This makes it easy to identify which stage failed.

7. Use timeout to prevent hung builds Always set a pipeline or stage-level timeout to ensure a hung process does not block an executor indefinitely.

8. Fail fast but report fully Use junit and jacoco post steps so test results and coverage are always published even when the build fails — this gives developers the information they need to fix problems quickly.

9. Use parallel stages for independent tasks

```
stage('Parallel Tests') {
  parallel {
    stage('Unit Tests') {
      steps { sh 'mvn test -Dtest=UnitTests' }
    }
    stage('Integration Tests') {
      steps { sh 'mvn test -Dtest=IntegrationTests' }
    }
  }
}
```

10. Tag Docker images with the build number, never only latest Using latest alone makes it impossible to roll back to a previous version. Always tag with BUILD_NUMBER or a version string in addition to latest.

Complete Production-Grade Jenkinsfile

```
pipeline {
    agent none

    options {
        timeout(time: 30, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '20'))
        disableConcurrentBuilds()
    }

    environment {
        APP_NAME = 'my-app'
        REGISTRY = 'myusername'
        IMAGE = "${REGISTRY}/${APP_NAME}"
    }

    stages {
        stage('Checkout') {
            agent { label 'linux' }
            steps {
                checkout scm
            }
        }

        stage('Build & Test') {
            agent {
                docker {
```

```
            image 'maven:3.9.4-eclipse-temurin-17'
            args '-v $HOME/.m2:/root/.m2'
        }
    }
    steps {
        sh 'mvn clean package -B'
    }
    post {
        always {
            junit 'target/surefire-reports/*.xml'
            jacoco execPattern: 'target/jacoco.exec'
        }
    }
}

stage('Build Docker Image') {
    agent { label 'linux' }
    steps {
        sh "docker build -t ${IMAGE}:${BUILD_NUMBER} ."
        sh "docker tag ${IMAGE}:${BUILD_NUMBER} ${IMAGE}:latest"
    }
}

stage('Push Docker Image') {
    agent { label 'linux' }
    when { branch 'main' }
    steps {
        withCredentials([usernamePassword(
```

```
            credentialsId: 'dockerhub-creds',
            usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_PASS'
        )]) {
            sh '''
                echo "$DOCKER_PASS" | docker login -u "$DOCKER_USER" --password-stdin
                docker push ${IMAGE}:${BUILD_NUMBER}
                docker push ${IMAGE}:latest
            '''
        }
    }
}

stage('Deploy to Staging') {
    agent { label 'linux' }
    when { branch 'main' }
    steps {
        sshagent(credentials: ['staging-server-key']) {
            sh """
                ssh -o StrictHostKeyChecking=no deploy@staging-server \
                "docker pull ${IMAGE}:${BUILD_NUMBER} && \
                 docker stop ${APP_NAME} || true && \
                 docker rm ${APP_NAME} || true && \
                 docker run -d \
                   --name ${APP_NAME} \
                   -p 8080:8080 \
                   --restart unless-stopped \
                   ${IMAGE}:${BUILD_NUMBER}"
```

```
            """
          }
        }
      }
    }

    post {
      success {
        mail to: 'team@example.com',
             subject: "SUCCESS: ${JOB_NAME} #${BUILD_NUMBER}",
             body: "Build and deployment succeeded.\n\nSee: ${BUILD_URL}"
      }
      failure {
        mail to: 'team@example.com',
             subject: "FAILED: ${JOB_NAME} #${BUILD_NUMBER}",
             body: "Build or deployment failed.\n\nSee: ${BUILD_URL}"
      }
      always {
        cleanWs()
      }
    }
}
```

Jenkins CI/CD Deployment Flow Summary

Step	Trigger	Action
Code pushed to feature branch	Webhook / pollSCM	Build + Test only

Step	Trigger	Action
Pull request opened	Webhook	Build + Test + Report results on PR
Merged to main	Webhook	Build + Test + Docker Build + Push to Registry + Deploy to Staging
Manual approval	User clicks in Jenkins UI	Deploy to Production
Build failure at any stage	Automatic	Email notification sent; subsequent stages skipped

---

End of Week 15 Notes

Complete Course Coverage Summary (Weeks 7–15)

Week	Unit	Topics Covered
Week 7	Unit 3	Microservices Architecture, Monolithic vs Microservices, Advantages, API Gateway
Week 8	Unit 3	Docker Compose YAML, Services, Volumes, Networks, Environment Variables, Build vs Image
Week 9	Unit 3	Secrets, Configs, Dependency Ordering, Use Case Deployments
Week 10	Unit 4	Build Tools, POM, Directory Structure, Build Lifecycle Phases
Week 11	Unit 4	Parent POM, Dependency Scopes, Transitive Dependencies, Version Conflicts, Plugins, Maven Wrapper
Week 12	Unit 4	Maven Docker Integration, Dockerfile for Maven Apps, dockerfile-maven-plugin, Pushing to Registries
Week 13	Unit 5	GitHub Actions Foundations, Triggers, Jobs, Steps, Actions, Runners
Week 14	Unit 5	Caching, Matrix Strategies, Multi-job Workflows, Docker CI, Pushing to Registries, Deployments

Week	Unit	Topics Covered
Week 15	Unit 6	Jenkins Architecture, Plugins, Security, Declarative Pipelines, Jenkinsfile, Docker + Maven Integration, CI/CD Flows